

GUIA PRÁTICO

Engenharia de Software com IA Generativa

Fundamentos, práticas e controles para desenvolver software com apoio de modelos generativos

Finalidade didática

Manual original em português do Brasil, preparado como base de conhecimento para um MVP de chatbot com RAG. O conteúdo foi estruturado com seções, procedimentos, exemplos e limites explícitos para facilitar recuperação semântica e avaliação.

Público: desenvolvedores, estudantes, líderes técnicos e profissionais que incorporam GenAI ao ciclo de desenvolvimento.

Versão: 1.0 - outubro de 2026

Material educacional. Não substitui políticas internas, avaliação jurídica, normas de segurança da informação ou revisão profissional.

Como usar este guia

Este manual parte de uma premissa: IA generativa é mais útil na Engenharia de Software quando atua como amplificador da capacidade da equipe, e não como substituta de requisitos, arquitetura, testes, revisão ou responsabilidade humana.

Para um chatbot RAG, cada capítulo funciona como uma unidade de conhecimento relativamente autônoma. Termos importantes aparecem de forma explícita e os procedimentos usam passos verificáveis.

Regra central

Não confunda geração plausível com evidência de correção. Código, documentação e decisões produzidos com IA devem passar por validação proporcional ao risco.

Sumário

1. O papel da IA Generativa na Engenharia de Software
2. Escopo e requisitos com apoio de IA
3. Engenharia de prompts para desenvolvimento
4. CO-STAR aplicado a assistentes de código
5. Desenvolvimento incremental com agentes e assistentes
6. Arquitetura e decisões técnicas
7. Geração, leitura e revisão de código
8. Testes, depuração e qualidade
9. Git, rastreabilidade e revisão de mudanças
10. RAG para conhecimento de software
11. Segurança, privacidade e contexto brasileiro
12. Avaliação de sistemas com GenAI
13. Operação, documentação e entrega
14. Anti-padrões e diagnóstico
15. Checklist prático
16. Glossário
17. Perguntas de validação para um RAG
18. Referências públicas brasileiras

1. O papel da IA Generativa na Engenharia de Software

IA generativa pode apoiar análise, desenho, programação, testes, documentação e investigação de falhas. Seu valor cresce quando recebe contexto suficiente, uma tarefa delimitada e critérios verificáveis de saída.

Onde ela ajuda

- Explorar alternativas e trade-offs.
- Transformar requisitos em tarefas menores e critérios de aceitação.
- Gerar rascunhos de código, testes, documentação e scripts repetitivos.
- Explicar código existente e investigar defeitos.
- Revisar mudanças procurando duplicação, acoplamento e lacunas de teste.

Onde ela não deve decidir sozinha

- Requisitos ambíguos ou conflitantes.
- Decisões irreversíveis de arquitetura ou segurança sem revisão.
- Uso de dados pessoais, sigilosos ou credenciais.
- Aprovação de código crítico apenas porque testes gerados pela própria IA passaram.
- Conclusões sobre conformidade legal sem análise competente.

2. Escopo e requisitos com apoio de IA

Antes de pedir código, defina o problema. Um assistente tende a preencher lacunas com suposições plausíveis; fronteiras explícitas reduzem retrabalho.

Um escopo mínimo deve responder

- Quem é o usuário?
- Qual problema será resolvido?
- Qual é a entrada?
- Qual é a saída esperada?
- O que está fora do escopo?
- Como saberemos que o MVP funciona?

Critérios de aceitação

- Prefira requisitos observáveis. Exemplo: uma pergunta vazia não deve ser enviada e a interface deve orientar o usuário.

3. Engenharia de prompts para desenvolvimento

Um prompt de engenharia deve reduzir ambiguidade. Contexto informa o estado; objetivo define a mudança; restrições impedem expansão; critérios tornam a saída verificável.

Estrutura prática

- Contexto: estado atual, arquivos e decisões.
- Tarefa: uma mudança específica.
- Restrições: o que não deve ser alterado.
- Critérios: como verificar a conclusão.
- Resposta: formato esperado.

Prompt ruim versus útil

- Ruim: “Crie um chatbot RAG completo.”
- Melhor: “Implemente somente o PDF Loader; preserve página; trate arquivo inexistente; não implemente chunking; execute testes e informe arquivos alterados.”

4. CO-STAR aplicado a assistentes de código

CO-STAR organiza instruções para GenAI como disciplina de comunicação entre desenvolvedor e assistente.

Elementos

- C - Context: projeto, estado, stack e restrições.
- O - Objective: mudança específica.
- S - Style: abordagem incremental, simples e testável.
- T - Tone: técnico, objetivo, didático ou crítico.
- A - Audience: destinatário do resultado.
- R - Response: formato esperado, como diff, checklist ou testes.

Exemplo

- C: Python 3.11 e pytest; loader existente. O: implementar chunking. S: simples, sem framework externo. T: técnico. A: desenvolvedor estudando RAG. R: código, testes e explicação.

5. Desenvolvimento incremental com agentes e assistentes

Assistentes podem alterar vários arquivos; isso aumenta produtividade e também o raio de impacto de um erro. Reduza o tamanho de cada tarefa.

Ciclo recomendado

- Inspecionar antes de alterar.
- Explicar o plano quando relevante.
- Modificar o menor conjunto possível de arquivos.
- Executar testes/verificações.
- Mostrar o que mudou.
- Revisar antes da próxima tarefa.
- Criar commit coerente ao final de etapa estável.

Instruções persistentes

- Um AGENTS.md pode registrar Python, comandos, convenções e limites arquiteturais. Ele funciona como guardrail, não como substituto dos requisitos.

6. Arquitetura e decisões técnicas

A IA pode propor arquiteturas convincentes demais. Pergunte qual problema cada camada resolve e qual custo adiciona.

Princípios

- Comece pelo fluxo de dados e responsabilidades.
- Evite abstração sem segundo caso real.
- Registre decisões difíceis de reverter.
- Separe UI, orquestração e infraestrutura quando isso melhorar testabilidade.
- Não transforme um MVP em microserviços sem necessidade.

ADR simplificado

- Registre contexto, decisão, alternativas, consequências e condições para revisão.

7. Geração, leitura e revisão de código

Código gerado deve ser tratado como código de um colaborador rápido que ainda precisa de revisão. Aparência profissional não prova adequação.

Checklist

- A função faz somente o requerido?
- Há dependências novas sem necessidade?
- Entradas inválidas são tratadas?
- Erros são ocultados?
- Há duplicação?
- Contratos existentes foram preservados?
- Há teste para o comportamento alterado?
- Logs expõem segredos?

Peça evidências

- Solicite arquivos alterados, testes executados, resultados, riscos restantes e hipóteses assumidas.

8. Testes, depuração e qualidade

IA acelera testes, mas testes gerados a partir do mesmo entendimento incorreto podem validar o erro. Combine níveis de teste e revisão dos critérios.

Estratégia mínima

- Teste feliz.
- Teste de borda.
- Teste de falha.
- Teste de integração.
- Smoke test ponta a ponta.

Depuração assistida

- Forneça erro completo, comando, versão do ambiente e trecho relevante. Peça hipóteses ordenadas e verificações discriminantes.

9. Git, rastreabilidade e revisão de mudanças

Git é ferramenta de segurança no desenvolvimento assistido por IA. Commits pequenos facilitam entender, reverter e comparar mudanças.

Fluxo recomendado

- Antes: git status.
- Depois: revisar git diff.
- Após testes e aprovação: commit relacionado ao objetivo.
- Não misture refatoração ampla com correção pequena.

Nunca versionar

- Segredos e tokens.
- Arquivos .env com credenciais.
- Ambientes virtuais.
- Caches e artefatos temporários.
- Dados sensíveis sem autorização.

10. RAG para conhecimento de software

RAG combina recuperação de informação com geração: antes de responder, o sistema procura trechos relevantes em uma base controlada e os fornece ao LLM como contexto.

Fluxo de ingestão

- PDF/documentos → extração → chunks → embeddings → índice vetorial.

Fluxo de consulta

- Pergunta → embedding → busca → Top-K chunks → prompt com contexto → LLM → resposta + fontes.

Chunking

- Chunks pequenos podem perder contexto; grandes podem misturar assuntos e consumir janela. Overlap preserva continuidade, mas aumenta redundância. Os valores devem ser avaliados no corpus real.

Embeddings e recuperação

- Embeddings representam textos como vetores. Similaridade semântica não garante que o trecho seja suficiente para responder.

Grounding e fontes

- Preserve documento, seção e página. Se o contexto não sustenta a resposta, declare insuficiência em vez de completar com conhecimento não autorizado.

Diagnóstico

- Se a resposta está errada, verifique primeiro se o trecho correto foi recuperado. Se não, investigue ingestão/chunking/embedding/retrieval; se sim, investigue prompt e geração.

11. Segurança, privacidade e contexto brasileiro

O uso de GenAI pode enviar prompts, código ou documentos a terceiros. Considere confidencialidade, política institucional, retenção, processamento e auditabilidade.

Dados e segredos

- Não cole chaves, senhas ou tokens em prompts.
- Não envie dados pessoais ou restritos a serviços não aprovados.
- Use dados sintéticos ou anonimizados quando apropriado.
- Revise logs antes de compartilhar.
- Mantenha segredos fora do repositório.

LGPD e governança

- Quando houver dados pessoais, avalie o tratamento sob a LGPD e políticas aplicáveis. Este guia não oferece parecer jurídico.

Prompt injection em RAG

- Trate conteúdo recuperado como dados, não como autoridade para alterar regras do sistema. Controle acesso, valide fontes e separe instruções de contexto.

12. Avaliação de sistemas com GenAI

Avaliar apenas se a aplicação roda é insuficiente. Sistemas generativos exigem critérios de comportamento, qualidade e segurança.

Assistentes de desenvolvimento

- Correção funcional.
- Aderência ao escopo.
- Qualidade dos testes.
- Necessidade das mudanças.
- Segurança.
- Clareza documental.

RAG

- Retrieval: trecho correto no Top-K?
- Suficiência: há evidência bastante?
- Grounding: resposta sustentada?
- Citação: fonte corresponde?
- Abstenção: recusa corretamente sem evidência?

13. Operação, documentação e entrega

Um projeto assistido por IA deve ser retomável sem o histórico do chat. A documentação do repositório é a fonte durável.

README

- Objetivo e escopo.
- Pré-requisitos.
- Instalação/configuração.
- Como executar.
- Como testar.
- Arquitetura.
- Limitações.
- Troubleshooting.

Entrega

- Execute testes, smoke test, comandos do README e git status; verifique credenciais ou dados indevidos no diff.

14. Anti-padrões e diagnóstico

Os anti-padrões abaixo aparecem com frequência em projetos assistidos por IA.

Evite

- Prompt gigante para construir tudo.
- Aceitar arquitetura sem questionar.
- Instalar dependências para cada problema.
- Confiar apenas no “funcionou” do assistente.
- Misturar ambientes Python.
- Testar somente a resposta final do RAG.
- Enviar dados sensíveis por conveniência.

15. Checklist prático

Use este checklist antes de encerrar uma etapa.

Verificação

- Problema e usuário claros.
- Escopo e fora do escopo registrados.
- Prompts com objetivo verificável.
- Inspeção antes de alteração.
- Mudanças pequenas.
- Dependências justificadas.
- Segredos protegidos.
- Testes relevantes.
- git diff revisado.
- Arquitetura documentada.
- Retrieval e geração avaliados separadamente.
- Abstenção tratada.
- README reproduzível.
- Limitações documentadas.
- Aprovação humana final.

16. Glossário

Agente: Sistema que usa um modelo para decidir ou executar ações com ferramentas e contexto.

Chunk: Trecho usado como unidade de indexação e recuperação.

Embedding: Representação vetorial usada para comparação semântica.

Grounding: Vinculação da resposta a evidências fornecidas ao modelo.

Alucinação: Conteúdo plausível não sustentado pela evidência necessária.

LLM: Large Language Model, modelo de linguagem de grande escala.

Prompt: Entrada com instruções e contexto.

Prompt injection: Instruções maliciosas que tentam desviar o sistema.

RAG: Retrieval-Augmented Generation; recuperação antes da geração.

Retriever: Componente que recupera conteúdo relevante.

Top-K: Quantidade máxima de resultados recuperados.

Vector Store: Estrutura para indexar vetores e buscar similaridade.

17. Perguntas de validação para um RAG

Estas perguntas facilitam a criação de um conjunto inicial de avaliação; suas respostas estão no próprio manual.

1. Qual é a principal regra ao usar código gerado por IA?
2. Por que tarefas pequenas reduzem o risco?
3. Qual a função de um AGENTS.md?
4. Quais são os seis elementos do CO-STAR?
5. Por que “crie um chatbot RAG completo” é inadequado?
6. Qual a diferença entre ingestão e consulta em RAG?
7. O que investigar primeiro quando uma resposta RAG está errada?
8. Qual o efeito de chunks muito pequenos e muito grandes?
9. Por que relevância não significa suficiência?
10. O que fazer quando o contexto não sustenta a resposta?
11. Por que git diff é útil?
12. Como reduzir o risco de instalar pacote no Python errado?
13. Que dados não devem ir a ferramentas externas não aprovadas?
14. Por que testes gerados pela IA não devem ser a única evidência?
15. Quais dimensões mínimas avaliar em um RAG?

Perguntas fora da base

Para testar abstenção: “Qual é o prazo legal brasileiro para auditoria de código gerado por IA?” e “Qual GPU específica é obrigatória para executar um RAG?”. Este guia não estabelece essas respostas.

18. Referências públicas brasileiras

Este manual é conteúdo didático original. As referências abaixo foram usadas como contexto e inspiração; o texto não reproduz integralmente esses materiais.

Governo Digital - GuIA Unificado de Inteligência Artificial para o Setor Público
Ciclo de vida, governança e verificações de conformidade.

Governo Digital - Caixa de Ferramentas do GuIA
Materiais para IA generativa, bases de conhecimento e RAG: arquitetura, chunking, indexação, métricas, riscos, segurança e LGPD.

Governo Digital - Guia de IA Generativa
Conceitos, riscos, boas práticas, escolha de ferramentas, governança de dados e supervisão humana.

Governo Digital - Guia de Introdução ao RAG no Setor Público
Recuperação vetorial, aumento de contexto, geração fundamentada e conferência de fontes.

Legislação, normas internas e políticas devem ser consultadas em suas fontes oficiais e versões vigentes.

Conclusão

Engenharia de Software com IA Generativa não elimina as disciplinas clássicas da engenharia. Ela aumenta a importância de escopo claro, arquitetura proporcional, testes, versionamento, segurança, documentação e revisão.

Padrão recomendado: contextualize, limite, implemente em pequenos passos, verifique evidências e mantenha responsabilidade humana.

Para o MVP RAG

Este documento foi escrito para que o chatbot responda dúvidas concretas sobre desenvolvimento com GenAI e permita avaliar retrieval, grounding, fontes e abstenção.